Circuitos de Alimentación y Niveles Lógicos

Clase 5 — I2C · SPI · Reguladores · Compatibilidad lógica

Presiona espacio para continuar →

Contenido

1. Circuitos de Alimentación y Niveles Lógicos
2. Contenido
3. Niveles Lógicos
4. Umbrales de Voltaje — La Zona Indeterminada
5. El Pin Flotante
6. Valores de Pull-Up, Internos y Compatibilidad 3.3V/5V
7. I2C — El Bus Físico
8. Open-Drain — Por Qué I2C Necesita Pull-Ups
9. I2C — Arquitectura Maestro/Esclavo
10. I2C — Las Direcciones de 7+1 Bits
11. I2C — Dirección Configurable y Conflictos
12. I2C — Librería Wire de Arduino
13. SPI — El Bus de 4 Cables
14. SPI — N Dispositivos en el Bus
15. SPI vs I2C — Comparativa
16. Circuitos de Alimentación
17. 3.1 Fundamentos de Alimentación
18. Fuentes Comunes en IoT
19. Regulación de Voltaje
20. Eficiencia y Calor — Intuición Práctica
21. Cómo Elegir el Regulador — Checklist
22. Baterías Li-ion / LiPo — Lo Esencial
23. Módulos de Carga
24. BMS y Seguridad — No Negociables
25. Medir Nivel de Batería con ESP32

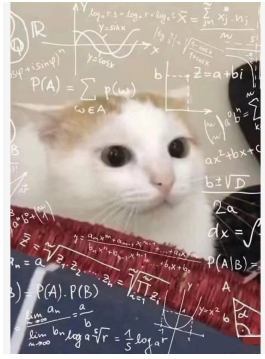
Niveles Lógicos

1

Lógico HIGH

Voltaje cercano a VCC

≈ 3.3V en ESP32



0

Lógico LOW

Voltaje cercano a GND

≈ 0V en ESP32

Los microcontroladores no hablan en 1s y 0s abstractos — hablan en **voltajes**. Entender los rangos es crítico para integrar sensores y módulos correctamente.

Umbrales de Voltaje — La Zona Indeterminada

HIGH ≥ 2.0V	**HIGH (lógico 1)** El ESP32 interpreta como 1 cualquier voltaje **por encima de 2.0V**. El margen garantiza que el ruido no cause errores en este rango.
⚠ ZONA GRIS 0.8–2.0V	**⚠ Zona Indeterminada (0.8V – 2.0V)** El pin puede leer 0 o 1 **aleatoriamente**. Un pin flotante sin conexión definida cae exactamente aquí — comportamiento impredecible garantizado.
LOW ≤ 0.8V	**LOW (lógico 0)** El ESP32 interpreta como 0 cualquier voltaje **por debajo de 0.8V**. Voltaje cercano a GND, estado bien definido.

No es blanco o negro — hay **rangos tolerados**. Esto se llama *noise margin* (margen de ruido). Todo el diseño de pull-ups y level shifters apunta a mantener las líneas fuera de la zona gris.

El Pin Flotante

¿Qué es un pin flotante?

Un pin **no conectado a ningún nivel definido** — ni a VCC ni a GND. Su voltaje queda a merced del entorno:

- Ruido electromagnético del ambiente
- Campos eléctricos de cables cercanos
- El roce de un dedo puede cambiar su voltaje

¿Dónde cae su voltaje?

En la **zona indeterminada** (0.8V–2.0V). En I2C: tramas corruptas, dispositivos que no responden, comunicaciones erráticas.

Solución: forzar un nivel definido por defecto

Pull-Up	Pull-Down
Resistencia a VCC → defecto **HIGH**	Resistencia a GND → defecto **LOW**

¿Cuándo usar cada una?

- **Pull-up:** I2C, botones activos en LOW, UART en reposo
- **Pull-down:** botones activos en HIGH, señales de enable (pines que activan un sensor u despiertan otro micro)

Valores de Pull-Up, Internos y Compatibilidad 3.3V/5V

Elección del valor

Valor	Resultado
`100 kΩ`	Flancos lentos → bits corruptos
`4.7 kΩ` ✓	Ideal — Standard 100 kHz
`2.2 kΩ` ✓	Fast mode 400 kHz o cables largos
`100 Ω`	Corriente excesiva → daño

Más velocidad o más metros → resistencia más baja

Pull-Ups Internos del ESP32

~**45 kΩ** — activables con `INPUT_PULLUP` o vía Wire automáticamente

⚠ Demasiado alto para I2C en producción. OK para botones o prototipado muy simple.

⚡ **Compatibilidad 3.3V vs 5V**

El ESP32 es un sistema de **3.3V**. Algunos módulos I2C de Arduino operan a **5V**.

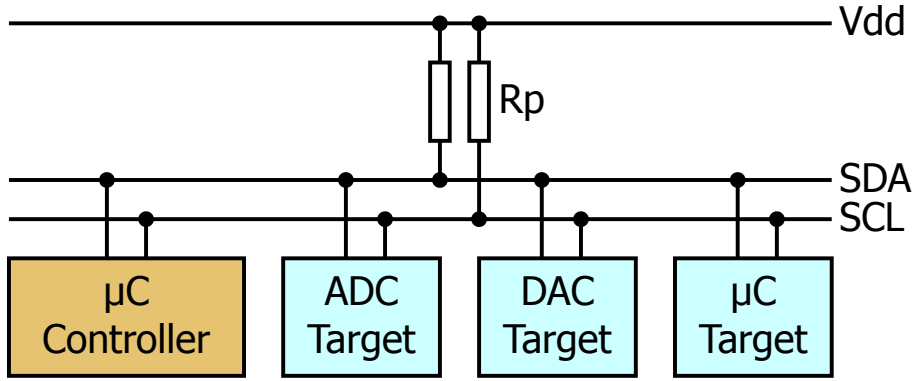
Peligro: 5V directo al ESP32 supera el límite ~3.6V de sus pines → daño permanente.

Solución: Level Shifter Bidireccional (BSS138) — convierte entre 3.3V y 5V en ambas direcciones.

Módulos modernos (OLED, BME280, MPU6050 breakouts) ya incluyen regulador → compatibles con 3.3V directamente.

I2C — El Bus Físico

- **SDA** (Serial Data) — datos en ambas direcciones
- **SCL** (Serial Clock) — reloj, siempre lo genera el maestro
- Solo **2 cables** para todos los dispositivos



Bus open-drain — pull-ups obligatorios
SDA y SCL son *open-drain*: los dispositivos solo pueden poner la línea en LOW. La resistencia pull-up es el único mecanismo para volver a HIGH — sin ella el bus no funciona.

Conexión en ESP32

`SDA → GPIO + 4.7 kΩ → 3.3V`
`SCL → GPIO + 4.7 kΩ → 3.3V`

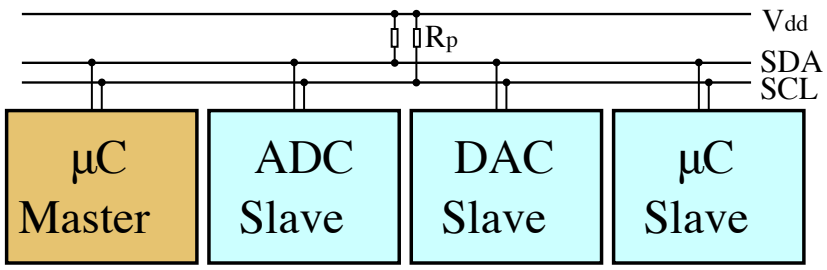
Una resistencia por línea, para todos los dispositivos

Pines I2C en ESP32-S3

SDA: GPIO 8 · SCL: GPIO 9

Configurables: `Wire.begin(SDA_PIN, SCL_PIN)`

Open-Drain — Por Qué I2C Necesita Pull-Ups



Open-drain: cada dispositivo tiene un transistor que *solo* puede conectar la línea a GND. Nadie puede empujar activamente a HIGH.

Sin pull-up → imposible

La línea puede bajar a LOW, pero **nunca vuelve a HIGH** — nadie la empuja hacia arriba. Comunicación completamente imposible.

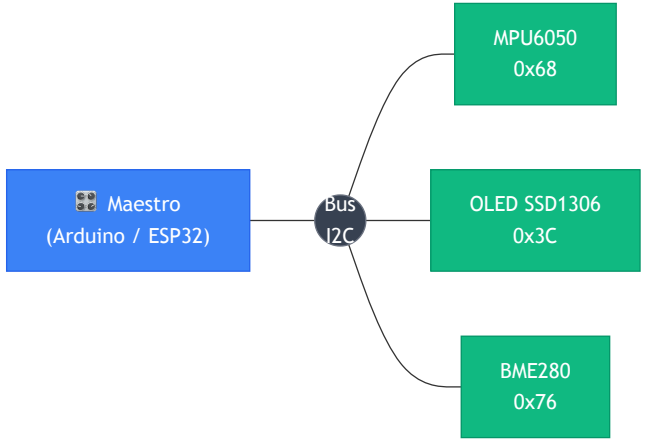
Con pull-up → funciona

La resistencia devuelve la línea a HIGH cuando ningún dispositivo la jala. Transmitir 0: jalar a GND. Transmitir 1: soltar (la resistencia sube).

Ventaja: bus compartido sin conflicto

Si dos dispositivos jalaran a la vez (ambos transmiten 0), no hay cortocircuito — ambos drenan a GND sin pelear.

I2C — Arquitectura Maestro/Esclavo



Maestro

- Siempre inicia la comunicación
- Genera el reloj (SCL)
- "Llama" al esclavo por su dirección

Esclavo

- Solo responde cuando se lo llama
- Tiene una dirección única en el bus
- Confirma que escuchó con un bit ACK

I2C — Las Direcciones de 7+1 Bits

A6	A5	A4	A3	A2	A1	A0	R/W

← 8 bits en el bus físico

Dirección (7 bits)

- 2^7 = **128** posibles (0x00–0x7F)
- ~16 reservadas → **~112 disponibles**
- Por convención: expresadas en **hex**
- Ejemplos: `0x68`, `0x3C`, `0x76`

Bit R/W — La Confusión

- **0** = Escribir al esclavo
- **1** = Leer del esclavo
- ⚠ Algunos datasheets muestran la dirección como 8 bits (con R/W incluido)
- MPU6050: datasheet dice `0xD0` → Wire usa `0x68`

Regla: en Wire siempre se usa la dirección de **7 bits**. Si el datasheet dice `0xD0`, dividilo entre 2 → `0x68`.

I2C — Dirección Configurable y Conflictos

Configuración por Hardware

Muchos módulos tienen pines `AD0`, `AD1` o `ADDR` que permiten cambiar 1–2 bits de la dirección con un jumper o conectando a VCC/GND.

```
AD0 = GND → 0x68
AD0 = 3.3V → 0x69
```

→ 2 MPU6050 en el mismo bus

⚠ Conflicto de Dirección

Si dos dispositivos tienen la misma dirección fija sin pin de configuración, no pueden coexistir.

Solución: multiplexor I2C `TCA9548A` — crea 8 sub-buses independientes, cada uno con su propio set de dispositivos.

🔍 I2C Scanner

Sketch que recorre todas las direcciones e imprime cuáles responden — indispensable para depurar.

```
#include <Wire.h>
void setup() {
  Wire.begin();
  Serial.begin(115200);
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    byte err = Wire.endTransmission();
    if (err == 0) {
      Serial.print("Dispositivo en 0x");
      Serial.println(addr, HEX);
    }
  }
}
void loop() {}
```

I2C — Librería Wire de Arduino

Funciones clave

Función	Acción
`Wire.begin()`	Inicia como maestro
`Wire.beginTransmission(addr)`	Abre sesión
`Wire.write(byte)`	Envía un byte
`Wire.endTransmission()`	Cierra + STOP
`Wire.requestFrom(addr, n)`	Solicita n bytes
`Wire.read()`	Lee un byte

Velocidades: Standard `100 kHz` (defecto) · Fast `400 kHz`
`Wire.setClock(400000);` — antes de Wire.begin()

```
#include <Wire.h>
#define MPU_ADDR 0×68

void setup() {
  Wire.begin();
  Serial.begin(115200);
  // Despertar el MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0×6B); // PWR_MGMT_1
  Wire.write(0×00); // Quitar sleep
  Wire.endTransmission();
}

void loop() {
  // Leer acelerómetro eje X
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0×3B);
  Wire.endTransmission(false); // Repeated START
  Wire.requestFrom(MPU_ADDR, 2);
  int16_t ax = (Wire.read() << 8) | Wire.read();
  Serial.println(ax);
  delay(100);
}
```

SPI — El Bus de 4 Cables

Serial Peripheral Interface — desarrollado por Motorola. Común en pantallas TFT, tarjetas SD, ADCs rápidos y sensores de alta velocidad.

Pin	Nombre completo	Dirección	Descripción
MOSI	Master Out Slave In	Maestro → Esclavo	Datos que el maestro envía al esclavo
MISO	Master In Slave Out	Esclavo → Maestro	Datos que el esclavo devuelve al maestro
SCK	Serial Clock	Maestro → Esclavo	Señal de reloj que sincroniza la transferencia
CS / SS	Chip Select / Slave Select	Maestro → Esclavo	Activa el esclavo deseado poniéndolo en LOW

Full-duplex: MOSI y MISO operan simultáneamente — envío y recepción al mismo tiempo

Sin direcciones: CS bajo (LOW) = dispositivo activo. Sin arbitraje ni ACK.

SPI — N Dispositivos en el Bus

Cada esclavo tiene su propio pin **CS**. El maestro activa solo uno a la vez poniéndolo en `LOW`.

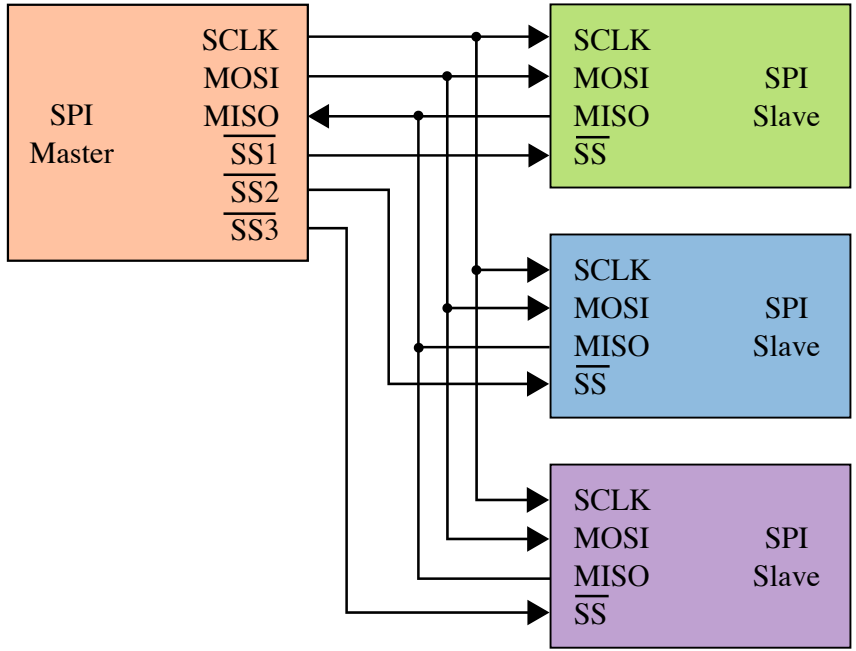
Pines GPIO necesarios

```
MOSI + MISO + SCK = 3 fijos
+ 1 CS por cada esclavo
```

Para N dispositivos: 3 + N pines

Comparación con I2C:
I2C: siempre 2 pines sin importar cuántos dispositivos
SPI: crece 1 pin por cada esclavo nuevo

Velocidad: 1–80 MHz (vs I2C 100–400 kHz) — hasta **800× más rápido**



SPI vs I2C — Comparativa

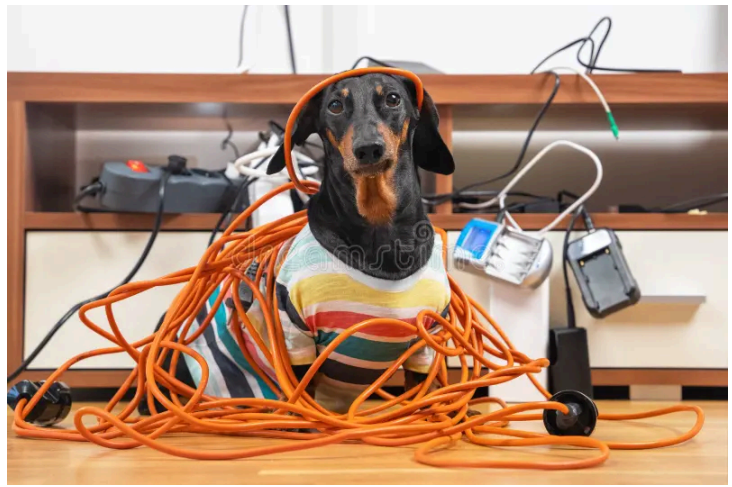
I2C

Característica	Valor
Cables de señal	**2** (SDA + SCL)
Pines para N dispositivos	**2** (siempre)
Velocidad	100 – 400 kHz
Dúplex	Semi (alternado)
Selección de esclavo	Dirección 7 bits
Pull-up	**Obligatorio**
Distancia máx.	~1 m
Protocolo	ACK · START · STOP

SPI

Característica	Valor
Cables de señal	**4** (MOSI·MISO·SCK·CS)
Pines para N dispositivos	**3 + N**
Velocidad	1 – 80 MHz
Dúplex	**Completo** (simultáneo)
Selección de esclavo	Pin CS dedicado
Pull-up	No necesario
Distancia máx.	~30 cm (PCB)
Protocolo	Simple (sin ACK)

Circuitos de Alimentación



3.1 Fundamentos de Alimentación

⚡ Voltaje — V

"Presión" que empuja la corriente. El ESP32 necesita exactamente **3.3 V** en sus pines lógicos. Más → daño permanente. Menos → no funciona. Se mide en **Volts (V)**.

〰 Corriente — A / mA

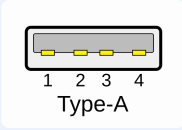
Cantidad de electricidad que fluye. ESP32 en reposo: **30–80 mA**. Transmitiendo WiFi: picos de **~240 mA**. Si la fuente no entrega esa corriente, el voltaje colapsa y el sistema falla. Se mide en **Amperes (A)** o **miliamperes (mA)**.

🔋 Potencia — P = V · I

Producto de voltaje y corriente. Útil para saber cuánto dura una batería y cuánto calor genera un regulador. ESP32 a 3.3 V × 100 mA = **0.33 W**. Se mide en **Watts (W)**.

La potencia disipada como calor en un regulador lineal es $\mathbf{P_{dis} = (V_{in} - V_{out}) \times I}$. Con 5 V → 3.3 V a 200 mA → 0.34 W de calor extra en el chip regulador.

Fuentes Comunes en IoT



USB 5V

La más cómoda en desarrollo. Entrega 500 mA (USB 2.0) o 900 mA (USB 3.0). No es opción en campo sin cable.

Li-ion / LiPo

3.7 V nominal, recargables, buena densidad de energía. Las más usadas en portátiles con ESP32. Requieren circuito de carga y protección.

Packs AA (alcalinas)

1.5 V por celda — 3 en serie = 4.5 V. Fáciles de conseguir, no recargables, corriente limitada. El voltaje cae progresivamente con el uso.

Fuente DC externa

Adaptadores de pared. Para proyectos fijos con alta demanda: motores, muchos LEDs, relés.

Regulación de Voltaje

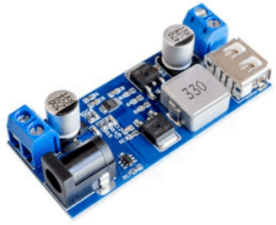
LDO — Low Dropout

"Quema" el voltaje sobrante como calor. **Entrada > Salida** siempre. Eficiencia baja: `(Vin−Vout)×I` se disipa. Ideal para corrientes bajas (<200 mA) y prototipos rápidos.

Ej: 5V→3.3V a 100mA → 170mW de calor. Típico: **AMS1117-3.3** en los módulos ESP32.

Buck — Reductor

Convierte voltaje alto → bajo por conmutación. **Eficiencia 85–95%**. Necesita inductor. Indispensable cuando la diferencia de voltaje es grande o la corriente es alta.



Buck Converter

Boost — Elevador

Convierte voltaje bajo → alto. Útil cuando la fuente cae bajo lo requerido durante descarga. Ej: LiPo a 3V → 3.3V estables hasta el final de la batería.

Boost Converter

Eficiencia y Calor — Intuición Práctica

- **LDO:** toda la diferencia **(Vin – Vout) × I** se convierte en calor. Límite práctico sin disipador: **500 mW** — superarlo requiere reemplazar por un buck.
- **Prueba del tacto:** si el regulador está caliente después de **30 segundos**, hay un problema → reducir ΔV, reducir corriente, o cambiar a buck.
- **Buck:** el voltaje sobrante no se quema, se recicla por conmutación. Eficiencia típica **85–95%**, el chip permanece frío incluso a corrientes altas.

Ejemplo LDO: 12V → 3.3V a 200mA

P_{dis} = (12 – 3.3) × 0.2 = 1.74 W de calor → necesita disipador o reemplazar por buck

Ejemplo Buck: 12V → 3.3V a 200mA al 90%

P_{dis} ≈ (1 – 0.9) × 12 × 0.2 = 0.24 W de calor → sin disipador

Cómo Elegir el Regulador — Checklist

- **¿Cuánta corriente máxima?** — El regulador debe soportarla con margen **×1.5**. Sistema a 200 mA pico → regulador de mínimo 300 mA.
- **¿Importa la duración de batería?** — Si sí, descartar LDO. Un LDO 12V→3.3V destruye hasta el 72% de la energía como calor.
- **¿Qué tan simple debe ser el diseño?** — LDO = 3 componentes (chip + 2 caps). Buck = chip + inductor + caps mínimo. LDO gana en simplicidad.
- **¿Cuánto espacio disponible?** — LDO es más compacto. El inductor del Buck ocupa espacio considerable en un PCB pequeño.
- **¿La fuente puede caer bajo Vout?** — Si la batería se puede descargar por debajo del voltaje requerido → Boost o Buck-Boost obligatorio.

Regla para este curso: LDO (AMS1117) para prototipos en escritorio con USB. Buck o Buck-Boost para proyectos con batería en campo donde la duración importa.

Baterías Li-ion / LiPo — Lo Esencial

Tensión nominal: 3.7V

El voltaje "promedio" de trabajo. Se usa para calcular energía total: **E = V × Ah**.

Carga completa: 4.2V

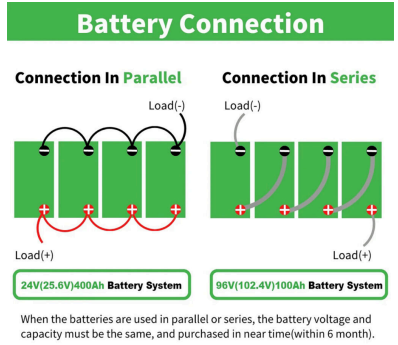
Voltaje máximo absoluto. Superarlo degrada la celda y puede causar **hinchamiento o incendio**.

Corte por descarga: 3.0V

Voltaje mínimo. Descargar por debajo daña la celda **irreversiblemente** y puede volverla peligrosa.

Duración de batería

t = Capacidad (mAh) ÷ Consumo (mA). LiPo 1000mAh con ESP32 activo a 80mA → ~12h. Transmitiendo WiFi continuamente → mucho menos.



Serie: suma voltajes, misma capacidad. **Paralelo:** suma capacidades, mismo voltaje.
Nunca conectar celdas de diferente capacidad o estado de carga.

Módulos de Carga

TP4056 — el más común

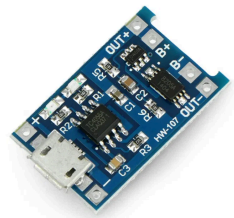
Carga a **1A** desde USB 5V. Incluye LED indicador (rojo = cargando, azul = completo). La versión **con DW01** integra protección de sobrecargas y sobredescarga — la única recomendable para principiantes.

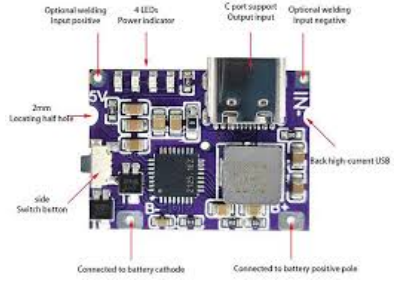
Módulos estilo power bank (IP5310)

Integran carga + protección + boost en un solo módulo. Permiten **cargar y usar simultáneamente**. Más caros pero más completos. Ideales para proyectos que deben seguir funcionando mientras se recarga.

⚠ Sin protección = peligro

Cargador sin protección + celda sin BMS es una **combinación peligrosa** para principiantes. Siempre usar módulos con protección integrada.





BMS y Seguridad — No Negociables

Protección sobrecarga — Corta si la celda supera 4.2V

Protección sobredescarga — Corta si baja de 3.0V

Protección cortocircuito — Corta la corriente instantáneamente

✅ Verificar polaridad **antes** de conectar. LiPo al revés = incidente inmediato
✅ No cargar sin supervisión con módulos desconocidos
✅ Usar conectores estandarizados (**JST-PH 2mm**)
✅ Cables al calibre correcto: 1–2A → AWG 24 mínimo
✅ Fusible o PTC en la línea positiva

🔥 **Almacenamiento seguro**

Guardar LiPo al **~50% de carga (3.8V)** si no se usa por días. En superficie no inflamable. Nunca en bolsillos o mochilas sin estuche rígido.

⛔ **Nunca usar una LiPo**

que esté hinchada o deformada, con la envoltura dañada, que haya recibido un golpe fuerte, o que no tenga BMS integrado.

Medir Nivel de Batería con ESP32

Divisor Resistivo + ADC

La forma más simple. Dos resistencias reducen el voltaje de la batería al rango del ADC del ESP32 (0–3.3V).

Por software se mapea el voltaje medido a porcentaje de carga.

Limitaciones: el ADC del ESP32 no es lineal, tiene ruido, y la curva de descarga de LiPo no es lineal → la lectura es una **aproximación**, no un valor preciso.

```
V_bat = V_adc × (R1 + R2) / R2
% ≈ map(V_bat, 3.0, 4.2, 0, 100)
```

Módulo Fuel Gauge (MAX17043)

Mide el estado de la batería por **impedancia** — mucho más preciso que el ADC.

Se comunica por **I2C** con el ESP32. Entrega directamente el porcentaje de carga y alerta cuando el nivel es crítico.

Para proyectos donde el nivel de batería importa de verdad: dispositivos portátiles, alertas de batería baja, UPS.

✅ La solución profesional. Más caro, más confiable.